

TLS401 Demo 常规业务代码逐段逐行解析

使用方法：左侧打开源码，右侧打开本文。本文的“行号”对应当前仓库版本；空行、纯括号和重复样板代码会与相邻语句合并讲解，但每个有业务含义的代码段都会覆盖。

排除范围：真实设备实现、厂商 SDK、RFID 字节协议、依赖探针和所有测试。设备接口作为业务边界保留。

1. src/domain/models.h

模块总览

该文件定义全项目共享的领域数据。它不负责流程，而负责让各层对“设备状态、标签、孔图、轮次、舱室”使用同一种语言。

第 1~9 行：头文件保护与依赖

```
#pragma once

#include <QDateTime>
#include <QImage>
#include <QSize>
#include <QString>
#include <QVector>
#include <optional>
```

- 第 1 行防止同一编译单元重复包含本头文件。
- 第 4~8 行分别为时间、图像、尺寸、文本和动态数组类型提供完整定义。
- 第 9 行引入 `std::optional`，用于表达舱室“有/无标签资料”，而不是用空字符串暗示状态。

第 11~14 行：三个状态枚举

```
enum class DeviceState { Offline, Ready, Busy, Error };
enum class RfidError { None, NoTag, InvalidPayload, DuplicateUid, Cancelled };
enum class WellState { Empty, Current, Complete, RetakeRequired };
```

- 第 11 行把设备状态压缩成 UI 和业务真正关心的四态。底层 SDK 的大量错误码应在设备适配层转换，不向上泄漏。
- 第 13 行描述一次 RFID 识别的业务结果。`None` 表示无错误，不是“没有结果”。
- 第 14 行描述孔位的显示状态。它不是数据库字段，而是由轮次 history 推导出来的状态。
- `enum class` 不允许隐式转成整数，也不会把 `Ready` 等名字污染到全局作用域。

第 16~24 行：标签资料

```
struct TagProfile {
    QString uid;
    QString dishNumber;
    QDateTime inseminationTime;
    QString femaleName;
    QString medicalRecordNumber;
    QString maleName;
    QDateTime identifiedAt;
};
```

- 第 17 行 `uid` 是标签技术标识，也是本地关联历史的关键键。
- 第 18~21 行是培养皿及患者相关的 RFID 业务字段。
- 第 22 行 `maleName` 是本地补录字段；Repository 的 UPSERT 刻意不覆盖旧值。
- 第 23 行记录本次识别/绑定发生时间，不等于授精时间。

这里的结构体是可变值对象，没有构造校验。因此“UID 为空但对象存在”等非法组合仍可能出现，Service 必须补前置校验。

第 26~31 行：RFID 结果对象

```
struct RfidResult {
    RfidError error = RfidError::None;
    TagProfile profile;
    QString message;
    bool ok() const { return error == RfidError::None; }
};
```

- 第 27 行默认成功，调用方构造失败结果时必须主动改 `error`。
- 第 28 行成功时携带解析后的领域资料。
- 第 29 行携带可显示的说明或错误信息。
- 第 30 行封装成功判断，避免业务代码到处重复比较枚举。

第 33~42 行：一次孔位拍摄

```
struct WellCapture {
    QImage image;
    QSize sourceSize;
    QString stagedImagePath;
    QDateTime capturedAt;
    bool active = true;
    bool available = true;
    double exposureUs = 0;
    double gainDb = 0;
};
```

- 第 34 行通常保存缩小后的预览图；当前轮次未持久化时也可能暂存完整传入图。
- 第 35 行单独保存原始尺寸，避免缩略图覆盖原图分辨率事实。
- 第 36 行只在轮次未正式完成时使用，指向 staging JPEG。
- 第 37 行是拍照业务时间，决定文件命名和回放顺序。
- 第 38 行表示这张是否为该孔当前有效版本。
- 第 39 行表示物理文件是否存在且可解码。`active=true, available=false` 是合法组合，UI 应提示重拍。

- 第 40~41 行记录相机参数，属于结果可追溯元数据。

第 44~53 行：采集轮次

```
struct CaptureRound {
    int number = 0;
    bool finished = false;
    QVector<QVector<WellCapture>> history;
    qint64 persistentId = 0;

    CaptureRound() : history(16) {}
    CaptureRound(int roundNumber, bool isFinished, qint64 id = 0)
        : number(roundNumber), finished(isFinished), history(16),
        persistentId(id) {}
};
```

- 第 45 行是同一 UID 下的人类可读轮次号。
- 第 46 行区分活动轮次与完成轮次。
- 第 47 行外层下标是孔位，内层保存重拍历史。
- 第 48 行 0 表示尚未落库；正数对应 `capture_round.id`。
- 第 50~52 行无论用哪种构造方式，都强制创建 16 个孔槽。后续 `history[wellNo - 1]` 的安全性依赖这个不变量。

第 55~63 行：舱室与发育天数

```
struct ChamberModel {
    int number = 0;
    std::optional<TagProfile> profile;
    QVector<CaptureRound> rounds;
};

inline int developmentDays(const TagProfile &profile) {
    return profile.inseminationTime.isValid()
        ? qMax(0, static_cast<int>
        (profile.inseminationTime.daysTo(QDateTime::currentDateTime())))
        : 0;
}
```

- 第 56 行是 1~4 的舱号，但类型本身没有约束范围。
- 第 57 行空 `optional` 就是空舱。
- 第 58 行历史属于标签，但为页面访问方便挂在当前舱室模型下；标签换舱时 Service 会按 UID 重新加载。
- 第 61~63 行对有效授精时间计算整日差，并用 `qMax` 防止未来时间产生负发育天数。无效时间返回 0，UI 无法区分“真实第 0 天”和“数据无效”，产品化时宜返回 `optional/result`。

架构沉淀

模型清楚表达了主业务，但缺少强不变量。下一阶段可以引入 `ChamberNumber`、`wellNumber` 等值类型，并把“轮次完整性”等规则变成模型方法，减少 Service 中的散落判断。

2. `src/devices/deviceinterfaces.h`

模块总览

该文件是业务层与硬件层之间的端口。这里只讲契约，不进入驱动实现。

第 1~5 行：依赖

```
#pragma once
#include "domain/models.h"
#include <QObject>
```

接口信号直接使用领域模型，所以包含 `models.h`；继承 `QObject` 是为了使用 Qt 元对象和信号槽。

第 6~16 行：RFID 端口

```
class IRfidService : public QObject {
    Q_OBJECT
public:
    using QObject::QObject;
    virtual void initialize() = 0;
    virtual void recognize() = 0;
    virtual void cancel() = 0;
signals:
    void stateChanged(DeviceState state, const QString &message);
    void recognized(const RfidResult &result);
};
```

- 第 7 行让 moc 为该生成信号槽元数据。
- 第 9 行继承 `QObject(QObject *parent)` 构造方式。
- 第 10~12 行是命令端：初始化、发起一次识别、取消当前工作。
- 第 14 行是持续状态事件；第 15 行是某次识别的终局事件。
- 方法返回 `void` 体现异步语义：调用成功不代表识别成功，最终结果从信号返回。

第 18~36 行：相机端口

```
class ICameraService : public QObject {
    Q_OBJECT
public:
    using QObject::QObject;
    virtual void connectDevice() = 0;
    virtual void disconnectDevice() = 0;
    virtual void startPreview() = 0;
    virtual void stopPreview() = 0;
    virtual void setExposure(double microseconds) = 0;
    virtual void setGain(double db) = 0;
    virtual double exposure() const = 0;
    virtual double gain() const = 0;
    virtual void capture() = 0;
signals:
    void stateChanged(DeviceState state, const QString &message);
    void previewFrame(const QImage &image);
    void captured(const QImage &image);
    void captureFailed(const QString &reason);
};
```

- 第 22~25 行把连接生命周期与预览生命周期分开：设备可以已连接但未取流。
- 第 26~29 行参数设置和读取使用明确单位，优于无单位的裸 `double`。
- 第 30 行触发一次业务拍照。
- 第 33 行是高频预览；第 34/35 行是拍照成功/失败的互斥终局事件。

接口没有表达“capture 请求 ID”，因此设计默认同一时刻最多一个拍照请求。控制器用 `m_captureRequestPending` 主动维护这个不变量。

架构沉淀

这是依赖倒置的正确方向：业务依赖抽象，真实设备依赖并实现抽象。若让控制器支持依赖注入，这两个接口就能直接替换成 Mock 或录制帧实现。

3. `src/app/workflowservices.h`

模块总览

头文件声明两个应用服务和它们持有的状态。阅读头文件的重点不是语法，而是先推断“谁拥有数据、谁可以修改、谁会通知外界”。

第 6~29 行：舱室会话声明

```
class Repository;
class ImageFileStore;

class ChamberSessionService : public QObject {
    Q_OBJECT
public:
    explicit ChamberSessionService(QObject *parent = nullptr);
    void setRepository(Repository *repository) { m_repository = repository; }
    void restoreProfiles(const QVector<ChamberModel> &profiles);
    const QVector<ChamberModel> &chambers() const { return m_chambers; }
    int selectedChamber() const { return m_selected; }
    void selectChamber(int chamberNo);
    bool bindProfile(const TagProfile &profile, QString *error = nullptr);
    void clearSelected();
    bool clearAll(QString *error = nullptr);
    ChamberModel *selectedModel();
signals:
    void changed();
    void selectionChanged();
private:
    QVector<ChamberModel> m_chambers;
    int m_selected = 0;
    Repository *m_repository = nullptr;
};
```

- 第 6~7 行前置声明避免头文件引入完整仓储定义，降低编译耦合。
- 第 13 行是 setter 注入。指针非 owning，生命周期由控制器管理。
- 第 15 行只暴露 `const` 容器，外部不能直接 push/clear。
- 第 18、20 行用 `bool + error out parameter` 返回可预期业务失败。
- 第 21 行却返回可变内部指针，是一个封装缺口。

- 第 23 行表示任意内容变化；第 24 行专门表示选择变化。
- 第 27 行 `0` 代表尚未选舱。

第 31~74 行：采集 workflow 声明

```
struct PendingCapture {
    TagProfile profile;
    int chamberNo = 0;
    int roundNo = 0;
    int wellNo = 0;
    qint64 roundId = 0;
    bool retake = false;
    QDateTime capturedAt;
    double exposureUs = 0;
    double gainDb = 0;
    QImage image;
};
```

第 34~45 行定义一次待提交拍摄的完整上下文。它不是长期领域对象，而是跨异步边界的数据传输对象。复制 `TagProfile` 和 `QImage` 看似昂贵，但 Qt 类型采用隐式共享，未修改时通常只增加引用计数。

```
bool createRound(QString *error = nullptr);
bool finishRound();
void discardActiveRound();
bool selectwell(int wellNo);
bool prepareCapture(..., PendingCapture *pending, QString *error = nullptr);
bool commitCapture(const PendingCapture &pending, QString *error = nullptr);
```

- 第 48~51 行是轮次和选孔命令。
- 第 52~54 行把接收图像拆成准备与提交，允许提交前发生异步保存或上下文校验。
- 第 55~56 行两个 `acceptCapture` 是同步便捷封装，当前生产控制器未使用。
- 第 57~60 行是查询接口。
- 第 63 行通知一轮已完成，控制器据此提示用户。
- 第 65~67 行是内部导航算法。
- 第 68~72 行分别保存会话依赖、持久化依赖、当前孔和活动数据库 ID。
- 第 73 行集中处理完整轮次的持久化与补偿。

架构沉淀

从头文件就能看出 Service 同时管理内存状态和持久化协调。接口简洁，但 setter 注入允许“忘记注入时静默仅内存运行”，适合测试，却可能让生产环境把数据库故障误当正常降级。

4. `src/app/workflowservices.cpp`

模块总览

这里实现舱室绑定规则与 16 孔采集规则，是领域状态实际发生变化的位置。

第 1~7 行：初始化四舱

```
#include "workflowservices.h"
#include "storage/repository.h"
#include "storage/imagefilestore.h"

ChamberSessionService::ChamberSessionService(QObject *parent) : QObject(parent)
{
    for (int i = 1; i <= 4; ++i)
        m_chambers.push_back({i, std::nullopt, {}});
}
```

- `.cpp` 需要调用仓储方法，所以这里引入完整定义。
- 构造函数创建编号 1~4 的固定舱室，profile 为空、rounds 为空。
- 从此 `m_chambers[chamberNo - 1]` 成为全模块依赖的映射不变量。

第 8~14 行：恢复、选择和定位

```
void restoreProfiles(const QVector<ChamberModel> &profiles) {
    for (int i = 0; i < m_chambers.size() && i < profiles.size(); ++i)
        m_chambers[i] = profiles[i];
    emit changed();
}

void selectChamber(int chamberNo) {
    if (chamberNo >= 1 && chamberNo <= 4 && m_selected != chamberNo) {
        m_selected = chamberNo;
        emit selectionChanged();
        emit changed();
    }
}

ChamberModel *selectedModel() {
    return m_selected ? &m_chambers[m_selected - 1] : nullptr;
}
```

- `restoreProfiles()` 只复制两个数组共同范围，避免越界，但没有验证 `profiles[i].number == i + 1`。它属于旧的“启动时按舱恢复”接口，当前控制器不调用；现行恢复入口是后面的 `bindProfile()`。
- 选择相同舱室不重复发信号，减少无意义刷新。
- 选择变化先发专用信号，再发通用变化信号；工作流连接两个信号时会发生两次 `selectFirstPendingwell()`，虽无害但有重复。
- `selectedModel()` 将 1-based 业务编号转成 0-based 容器下标。

第 15~27 行：绑定标签

```

for (const auto &chamber : m_chambers)
    if (chamber.number != m_selected && chamber.profile
        && chamber.profile->uid == profile.uid) {
        if (error)
            *error = QStringLiteral("UID is already bound to chamber
%1.").arg(chamber.number);
        return false;
    }

```

第 16 行先在内存层检查 UID 唯一性。排除当前舱，是为了允许同一舱重新识别同一标签并刷新资料。

```

if (auto *model = selectedModel()) {
    QVector<CaptureRound> rounds;
    if (m_repository && !m_repository->loadRoundsForUid(profile.uid, &rounds,
error))
        return false;
    if (m_repository && !m_repository->assignChamber(model->number, profile,
profile.identifiedAt,
error))
        return false;
    model->profile = profile;
    model->rounds = std::move(rounds);
    emit changed();
    return true;
}

```

- 第 17 行没有选舱则跳到错误分支。
- 第 18~19 行按 UID 加载历史，而不是按舱加载，符合培养皿换舱后历史跟随标签的规则。**这就是当前应用重启后的数据恢复机制：再次扫描同一 UID，数据库中的已完成轮次便重新进入内存模型。**
- 第 20 行数据库绑定成功后才修改内存。
- 第 21~22 行用移动赋值接管轮次数组，避免不必要深拷贝。
- 第 26 行错误赋值和 `return false` 写在同一源码行，逻辑正确但可读性不佳。

第 28~29 行：清空舱室

```

void clearSelected() {
    if (auto *model = selectedModel()) {
        if (m_repository) {
            QString error;
            if (!m_repository->clearChamber(model->number, &error)) return;
        }
        model->profile.reset();
        model->rounds.clear();
        emit changed();
    }
}

```

数据库失败时保持内存不变是正确的，但错误被局部变量吞掉，调用 UI 无法提示用户。`clearAll()` 改为返回 bool 和 error，接口更完整。

```
bool clearAll(QString *error) {
    for (auto &model : m_chambers) {
        if (m_repository && !m_repository->clearChamber(model.number, error))
            return false;
        model.profile.reset();
        model.rounds.clear();
    }
    m_selected = 0;
    emit selectionChanged();
    emit changed();
    return true;
}
```

循环逐舱提交并逐舱改内存，没有总事务。第 N 舱失败会留下部分清空状态，这是需要修正的原子性边界。

第 31~33 行：工作流与会话联动

```
CaptureWorkflowService::CaptureWorkflowService(ChamberSessionService *sessions,
QObject *parent)
    : QObject(parent), m_sessions(sessions) {
    connect(sessions, &ChamberSessionService::changed, this, [this] {
        selectFirstPendingwell();
        emit changed();
    });
    connect(sessions, &ChamberSessionService::selectionChanged, this, [this] {
        selectFirstPendingwell();
    });
}
```

会话变化时工作流重新定位待拍孔，并把变化继续传播给 UI。这里没有检查 `sessions` 是否为空，构造契约要求必须传入有效指针。

```
CaptureRound *activeRound() {
    auto *model = m_sessions->selectedModel();
    if (!model || model->rounds.isEmpty() || model->rounds.last().finished)
        return nullptr;
    return &model->rounds.last();
}
```

只有“当前舱最后一轮未完成”才是活动轮次。更早的未完成轮次不会被发现，代码隐含了每个标签最多一个 in-progress 轮次的不变量，数据库部分唯一索引也做了对应约束。

第 33 行 `hasActiveRound() const` 使用 `const_cast`，只是为了复用非 `const` 查询，不会在当前实现修改对象，但 `const` 语义不够干净。

第 34~35 行：创建与手工完成轮次

```
bool createRound(QString *error) {
    auto *model = m_sessions->selectedModel();
    if (!model || !model->profile) { /* 报错 */ return false; }
    if (activeRound()) { /* 报错 */ return false; }
    int roundNo = static_cast<int>(model->rounds.size()) + 1;
```

```

    if (m_repository
        && !m_repository->nextRoundNumber(model->profile->uid, &roundNo, error))
        return false;
    m_activeRoundId = 0;
    model->rounds.push_back(CaptureRound(roundNo, false));
    m_currentWell = 1;
    emit changed();
    return true;
}

```

- 必须先有标签，轮次才能归属到确定 UID。
- 数据库轮次号覆盖内存推算值，防止只加载部分历史时重复。
- 当前阶段不插数据库，因此 `m_activeRoundId` 清零。
- 创建成功后 UI 会看到 1 号孔为 Current。

`finishRound()` 再次遍历 16 孔，要求每孔非空且最后一张 available，然后持久化并标记 finished。当前主流程在第 16 张提交时自动完成，所以该方法没有生产调用点。

第 36~50 行：丢弃活动轮次

```

auto *model = m_sessions->selectedModel();
auto *round = activeRound();
if (!model || !round) return;
if (m_imageStore) {
    for (const auto &well : round->history)
        for (const auto &capture : well)
            m_imageStore->remove(capture.stagedImagePath);
}
model->rounds.removeLast();
m_activeRoundId = 0;
m_currentWell = 1;
emit changed();

```

- 先取得两个内部指针；只要中途不修改 rounds，它们有效。
- 双层循环删除本轮所有版本的 staging 文件。
- `removeLast()` 与 `activeRound()` 的“只能是最后一轮”定义严格配套。
- 清理状态后通知 UI。

第 51 行：选择孔位

```

if (!activeRound() || wellNo < 1 || wellNo > 16) return false;
m_currentWell = wellNo;
emit changed();
return true;

```

只有活动轮次可选孔。它允许选择已经完成的孔，但普通拍摄会被 `prepareCapture` 拒绝，必须以 retake 模式提交。

第 52~69 行：准备拍摄

```
auto *round = activeRound();
auto *model = m_sessions->selectedModel();
if (!round || !model || !model->profile || image.isNull()
    || round->history.size() != 16) {
    /* 报错 */
    return false;
}
```

第 55 行把上下文、图像和容器结构一起校验。`round` 存在时 `model` 理论上必然存在，重复判断属于防御式编程。

```
auto &well = round->history[m_currentwell - 1];
if (!retake && !well.isEmpty()) return false;
if (retake && well.isEmpty()) return false;
if (!pending) return false;
```

- 普通拍摄不能覆盖已有结果。
- 重拍必须有旧结果。
- 输出容器不能为空。
- 代码没有显式重新验证 `m_currentwell` 范围，而是依赖所有修改入口保持 1~16。

```
pending->profile = *model->profile;
pending->chamberNo = model->number;
pending->roundNo = round->number;
pending->wellNo = m_currentwell;
pending->roundId = round->persistentId;
pending->retake = retake;
pending->capturedAt = QDateTime::currentDateTime();
pending->exposureUs = exposureUs;
pending->gainDb = gainDb;
pending->image = image;
```

第 58~67 行冻结所有提交所需上下文。这段复制是防止异步返回写错舱室/孔位的核心。

第 71~85 行：提交前再次验证

```
if (!round || !model || !model->profile
    || model->number != pending.chamberNo
    || model->profile->uid != pending.profile.uid
    || round->number != pending.roundNo
    || round->persistentId != pending.roundId
    || m_currentwell != pending.wellNo) {
    /* Capture context changed */
    return false;
}
```

逐项比较说明：舱不能变、标签不能换、轮次不能换、数据库 ID 不能换、当前孔不能换。只比较 `profile` 的 `UID` 而不是全字段是合理的，其他资料更新不应让同一次拍摄失效。

第 81~85 行再次校验目标孔的空/非空状态，防止 `prepare` 后已有其他路径修改该孔。

第 86~105 行：构造并暂存拍摄结果

```
wellCapture capture;
capture.capturedAt = pending.capturedAt;
capture.sourceSize = pending.image.size();
capture.active = true;
capture.available = true;
capture.exposureUs = pending.exposureUs;
capture.gainDb = pending.gainDb;
```

先构造局部对象，只有所有必要步骤成功才 push 到 history，避免容器出现半初始化项。

```
if (m_imageStore) {
    if (!m_imageStore->stageJpeg(..., &capture.stagedImagePath, error))
        return false;
    capture.image = pending.image.scaled(QSize(800, 600),
                                         Qt::KeepAspectRatio,
                                         Qt::FastTransformation);
} else {
    capture.image = pending.image;
}
```

- 有文件仓储时先落 staging；失败则内存不提交。
- 内存只保留最多 800x600 的预览，控制长期内存。
- 无仓储时保留原图，支持纯内存测试/演示模式。

```
for (auto &previous : well) {
    previous.active = false;
    if (m_imageStore) m_imageStore->remove(previous.stagedImagePath);
}
well.push_back(std::move(capture));
```

提交新版本前将旧版本失活并删旧 staging。普通拍摄时 well 为空，循环不执行；重拍时保留旧内存元数据但旧图文件已删。

第 106~117 行：完成判断和推进

```
bool completed = true;
for (const auto &item : round->history)
    if (item.isEmpty() || !item.last().available)
        completed = false;
```

只要任一孔为空或最后一张不可用，本轮未完成。没有提前 break 只多做极少量循环。

```

if (completed) {
    if (!persistCompletedRound(...)) return false;
    round->finished = true;
    emit roundCompleted();
} else {
    advanceToNext();
}
emit changed();
return true;

```

先持久化成功再标记完成。若持久化失败，新 capture 已在内存，调用者必须决定重试还是丢弃；当前控制器选择结束序列并丢弃活动轮次。

第 119~128 行：同步便捷入口

两个 `acceptCapture` 都是 `prepare + commit`，第二个再把曝光/增益补成 0。它们减少调用样板，但隐藏了两阶段边界，适合测试或真正同步的场景。

第 130~168 行：完整轮次持久化

```

if (!m_repository || !m_imageStore)
    return true;

```

任一依赖缺失就把持久化视为成功。这方便测试，但生产中数据库故障后不应默默把数据当完成。

```

qint64 roundId = 0;
if (!m_repository->createRound(profile, chamberNo, round->number,
                                &roundId, error))
    return false;

```

先创建 `in_progress` 数据库轮次，后续图片都引用该 ID。

```

for (int wellNo = 1; wellNo <= 16; ++wellNo) {
    wellCapture &capture = round->history[wellNo - 1].last();
    QString path;
    const QSize imageSize = capture.sourceSize.isValid()
        ? capture.sourceSize : capture.image.size();

```

- 这里依赖调用前已经确认 16 孔都非空。
- 优先记录原始尺寸，旧数据无 `sourceSize` 时退回预览尺寸。

```

const bool stored = capture.stagedImagePath.isEmpty()
    ? m_imageStore->savePng(...)
    : m_imageStore->finalizeStagedJpeg(...);

```

无 staging 的内存模式保存 PNG；正常生产流程有 staging，因此最终复制成 JPEG。这个分支导致实际格式取决于前置路径。

```

if (!stored || !m_repository->insertImage(...)) {
    if (!path.isEmpty()) paths.push_back(path);
    for (const auto &savedPath : paths) m_imageStore->remove(savedPath);
    m_repository->deleteRound(roundId, nullptr);
    return false;
}
paths.push_back(path);

```

每成功一个正式文件就记录路径。文件或 SQL 失败时删除本次已发布文件，再删除数据库轮次及其图片行。这是跨资源补偿。

```

if (!m_repository->finishRound(roundId, "completed", error)) {
    for (const auto &path : paths) m_imageStore->remove(path);
    m_repository->deleteRound(roundId, nullptr);
    return false;
}

```

只有 16 张图全部完成才把轮次状态改成 completed。失败补偿与循环内一致。

```

round->persistentId = roundId;
m_activeRoundId = roundId;
for (auto &well : round->history)
    for (auto &capture : well) {
        m_imageStore->remove(capture.stagedImagePath);
        capture.stagedImagePath.clear();
    }
return true;

```

最后回填数据库 ID，并清理所有 staging。m_activeRoundId 此后没有被其他业务逻辑读取，属于残留状态。

第 169~172 行：孔位导航与历史

```

for (int offset = 1; offset <= 16; ++offset) {
    const int candidate = (m_currentWell - 1 + offset) % 16;
    if (round->history[candidate].isEmpty()) {
        m_currentWell = candidate + 1;
        return;
    }
}

```

从下一孔开始环形扫描空位。% 16 让 16 号后回到 1 号。

selectFirstPendingWell() 从 0 开始寻找空孔或 unavailable 孔；找不到则回到 1。wellStates() 先生成 16 个 Empty，再将有图孔映射为 Complete/RetakeRequired，最后只把空的当前孔改为 Current。已完成且当前选中的孔仍显示 Complete。

historyForWell() 汇总每一轮该孔的所有内存版本；它不筛选 active/available。调用者如果用于展示，必须自行定义筛选规则。

架构沉淀

该模块最值得掌握的是上下文快照、提交前复核和跨数据库/文件的补偿事务。最需要改进的是 const 正确性、崩溃恢复、同步 I/O 和“缺持久化依赖也算成功”的隐式降级。

5. src/app/applicationcontroller.h

模块总览

控制器对外提供完整用例，对内持有各服务和序列状态。它是 UI 与业务/设备之间的唯一总协调者。

第 1~8 行：依赖

```
#include "app/workflowservices.h"
#include "devices/deviceinterfaces.h"
#include <QPointer>
#include <QSet>
#include <QThread>
#include <QVector>
```

头文件需要工作流和设备接口的完整类型；QPointer 用于观察 QObject 是否已被删除，QSet 记录本次序列已完成的舱号，QThread 为计划中的保存线程，QVector 保存舱室执行队列。

第 10~34 行：公开状态与命令

```
class ApplicationController : public QObject {
    Q_OBJECT
    enum class SequenceMode { None, Identifying, Capturing };
```

- SequenceMode 私有，UI 只通过 identifying()/sequenceActive() 查询，不依赖内部枚举。
- 三态互斥，避免“识别中且拍照中”的非法组合。

```
ChamberSessionService *sessions() { return &m_sessions; }
CaptureWorkflowService *workflow() { return &m_workflow; }
IRfidService *rfid() { return m_rfid; }
ICameraService *camera() { return m_camera; }
```

第 15~18 行暴露子服务指针。UI 因此能直接连接相机 preview，也能直接调用 startPreview()；便利的代价是控制器无法完全约束设备调用顺序。

```
DeviceState rfidState() const;
DeviceState cameraState() const;
bool databaseReady() const;
void initializeDevices();
void identify();
void identifyAll();
void startCaptureSequence();
```

- 第 19~21 行是状态快照。
- 第 22 行完成启动后设备连接。
- 第 23 行针对当前舱单次识别；第 24 行按 4→3→2→1 批量识别。

- 第 25 行启动批量采集。

```
bool sequenceActive() const { return m_sequenceMode != SequenceMode::None; }
bool identifying() const { return m_sequenceMode == SequenceMode::Identifying; }
bool chamberCompletedInCurrentCapture(int chamberNo) const {
    return m_completedCaptureChambers.contains(chamberNo);
}
bool canStartCapture() const { return m_captureSequenceAuthorized &&
!sequenceActive(); }
QString sequenceStatus() const { return m_sequenceStatus; }
bool captureSequencePaused() const { return m_capturePaused; }
```

第 27~32 行为 UI 提供派生查询。chamberCompletedInCurrentCapture() 只回答某舱是否在**当前这次自动采集**中完成，不能用历史 round.finished 替代；首页据此区分“历史有数据”和“本轮刚完成”。canStartCapture 不检查相机 Ready 或数据库 Ready，这些失败要到流程运行时才能暴露。

第 35~42 行：信号和内部步骤

```
signals:
    void message(const QString &text, bool error);
    void stateChanged();
private:
    void identifyNext();
    void captureNext();
    void finishSequence(const QString &message = {});
    void handleCapturedFrame(const QImage &image);
```

- message 把业务提示和错误统一交给 UI，bool error 简单但扩展性弱，可改严重级别枚举。
- stateChanged 是粗粒度重渲染信号。
- 四个私有方法构成状态机的推进、完成和相机回调节点。

第 43~69 行：成员状态

```
ChamberSessionService m_sessions;
CaptureWorkflowService m_workflow;
IRfidService *m_rfid;
ICameraService *m_camera;
```

会话和工作流按值持有，生命周期与控制器一致；设备是 QObject 子对象，由 parent 自动释放。

```
DeviceState m_rfidState = DeviceState::Offline;
DeviceState m_cameraState = DeviceState::Offline;
bool m_databaseReady = false;
bool m_devicesInitialized = false;
```

- 两个状态是设备信号的缓存，便于 UI 同步读取。
- 数据库 Ready 只在 open 成功后为真。
- 初始化标志让 initializeDevices() 幂等。

```
Database *m_database = nullptr;
Repository *m_repository = nullptr;
ImageFileStore *m_imageStore = nullptr;
```

这三个不是 QObject，没有 parent，析构函数必须显式按依赖逆序 delete。

```
QVector<int> m_sequenceChambers;
QSet<int> m_completedCaptureChambers;
int m_sequenceIndex = 0;
SequenceMode m_sequenceMode = SequenceMode::None;
bool m_captureRetake = false;
bool m_capturePaused = false;
bool m_previewFrameAvailable = false;
bool m_captureRequestPending = false;
bool m_captureRequestDispatched = false;
bool m_discardPendingCaptureResult = false;
bool m_captureSequenceAuthorized = false;
quint64 m_captureRequestToken = 0;
QPointer<QThread> m_saveThread;
QString m_sequenceStatus;
```

- 队列和索引共同表示当前舱；完成集合记录本次序列真正完成过的舱，避免历史完成轮次让 4、2 或 2、1 等队列提前结束。
- `m_captureRetake` 只服务手工 `captureCurrent()`。
- `paused` 是用户浏览详情造成的临时暂停。
- `preview available` 是采集启动守卫，一旦收到过帧就保持 `true`。
- `request pending` 是同一时刻只允许一个 `capture` 的软件锁。
- `dispatched` 区分“延时请求尚未调用相机”和“相机已经收到请求”。
- `discard flag` 表示用户暂停后，这次已经下发的拍照结果必须被消费但不能写入业务状态。
- `token` 使旧的 `QTimer::singleShot` 回调在恢复后失效，不能误下发新一次拍照。
- `authorized` 把“一轮 RFID 识别”和“一轮拍照”绑定，拍完后必须重新识别。
- `m_saveThread` 当前从未赋值，是未完成的异步保存设计。

架构沉淀

头文件展示了状态机的全部状态维度。评审这类类时，应把每个字段列成“初始值、谁写、谁读、结束时是否复位”表，能很快找出残留和不可达状态。

6. src/app/applicationcontroller.cpp

模块总览

该实现把设备事件转换为应用用例推进。理解它的关键是沿信号回调阅读，不要仅按文件从上到下看。

第 1~20 行：依赖和图像守卫

```
namespace {
bool isUsableCapture(const QImage &image)
{
    return !image.isNull() && image.width() >= 32 && image.height() >= 32;
}
}
```

- 匿名命名空间让辅助函数只在本编译单元可见。
- 注释说明暗或均匀画面可能是真实空孔，因此不能用亮度/方差判失败。
- 实际守卫只排除空图和极小图。后文错误消息却称“过暗或无细节”，语义不一致。

第 22~40 行：构造对象图与持久化

```
ApplicationController::ApplicationController(QObject *parent)
: QObject(parent),
  m_sessions(this),
  m_workflow(&m_sessions, this),
  m_rfid(nullptr),
  m_camera(nullptr)
```

成员按声明顺序构造。工作流拿到会话地址，并且二者都以控制器为 QObject parent。

```
m_rfid = new RealRfidService(this);
m_camera = new RealCameraService(this);
```

控制器直接依赖真实类，接口只用于后续访问。这阻断了构造注入，是测试性上的主要缺口。

```
const QString root =
QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
QDir().mkpath(root);
m_database = new Database;
QString dbError;
if (m_database->open(QDir(root).filePath("tls401.sqlite"), &dbError)) {
    m_databaseReady = true;
    m_repository = new Repository(m_database->connection());
    m_imageStore = new ImageFileStore(root);
    m_sessions.setRepository(m_repository);
    m_workflow.setPersistence(m_repository, m_imageStore);
} else {
    emit message(..., true);
}
```

- AppDataLocation 避免写程序安装目录，适合 Windows 非管理员运行。
- mkpath 的返回值未检查，后续数据库 open 会间接暴露失败。
- DB 成功后按依赖顺序创建仓储并注入。
- 构造函数内 emit message 通常早于 MainWindow 连接该信号，所以这条初始化失败消息可能无人接收；UI 只能从 databaseReady=false 看见异常状态。
- 这里没有调用 loadAssignments/loadRounds，因为当前产品采用“再次识别同一 UID 时恢复历史”的策略：bindProfile() 内部调用 loadRoundsForUid()。restoreProfiles() 与这两个按舱加载接口是旧的启动恢复方案遗留，不应被描述成现行主流程缺陷。

第 42~53 行：设备状态信号

```
connect(m_rfid, &IRfidService::stateChanged, this,
        [this](DeviceState state, const QString &detail) {
    m_rfidState = state;
    emit stateChanged();
    if (state == DeviceState::Error && !detail.isEmpty())
        emit message("RFID: " + detail, true);
});
```

相机连接同理。先更新缓存再通知 UI，保证槽函数读取的是新状态。错误状态额外转换成用户提示；非错误 detail 不展示。

第 54~75 行：拍照失败、预览和成功

```
connect(m_camera, &ICameraService::captureFailed, this,
        [this](const QString &reason) {
    if (!m_captureRequestPending) return;
    const bool discardResult = m_discardPendingCaptureResult;
    m_captureRequestPending = false;
    m_captureRequestDispatched = false;
    m_discardPendingCaptureResult = false;
    if (discardResult) {
        if (m_sequenceMode == SequenceMode::Capturing && !m_capturePaused)
            QTimer::singleShot(0, this, [this] { captureNext(); });
        return;
    }
    emit message(..., true);
    if (m_sequenceMode == SequenceMode::Capturing)
        finishSequence();
});
```

没有在途请求的失败信号视为迟到事件并忽略。正常失败会释放两种请求状态；如果这次请求已在暂停时标记为 discard，则只完成清理，并在恢复状态下从原队列继续。正常自动拍摄失败仍会终止序列，手工拍照只提示错误。

```
connect(m_camera, &ICameraService::previewFrame, this,
        [this](const QImage &) {
    m_previewFrameAvailable = true;
    if (m_sequenceMode == SequenceMode::Capturing && !m_capturePaused)
        captureNext();
});
```

第一个预览帧解除采集启动守卫。之后每一帧都可能调用 `captureNext()`，但 pending 锁会阻止重复请求。

`captured` 信号直接转给 `handleCapturedFrame`，把复杂逻辑从 Lambda 抽出。

第 76~102 行：RFID 识别结果

```

if (m_sequenceMode == SequenceMode::Identifying) {
    const int chamber = m_sequenceChambers.value(m_sequenceIndex);
    if (!result.ok()) {
        emit message(..., true);
    } else {
        QString error;
        if (!m_sessions.bindProfile(result.profile, &error))
            emit message(error, true);
    }
    ++m_sequenceIndex;
    QTimer::singleShot(600, this, [this] { identifyNext(); });
    return;
}

```

- 批量模式按当前索引确定结果归属舱室。value() 越界会返回 0，但正常状态机保证索引有效。
- 单舱失败不停止队列。
- 600 ms 是操作节奏延迟，不是后台线程。
- return 防止批量结果再进入单次识别分支。

普通模式下，失败直接提示；成功绑定后设置 m_captureSequenceAuthorized=true，通知 UI 启用拍照按钮，并显示设备结果消息。

第 103~105 行：轮次完成提示

工作流 roundCompleted 只产生“16 孔完成”消息。真正的舱室索引推进发生在 handleCapturedFrame，因此该信号不是序列推进依据。

第 109~122 行：析构收尾

```

if (m_camera) m_camera->disconnectDevice();
if (m_rfid) m_rfid->cancel();
if (m_saveThread) {
    m_saveThread->wait();
    m_saveThread = nullptr;
}
delete m_imageStore;
delete m_repository;
delete m_database;

```

- 先停止仍可能产生回调的设备，再等待保存线程，最后释放存储依赖。
- 设备由 QObject parent 自动删除，不手工 delete。
- 若设备断开是异步的，析构里没有等待其完成；真实实现必须保证对象销毁安全。
- saveThread 当前永远为空。

第 124~140 行：设备初始化与单舱识别

initializeDevices() 用标志防止重复初始化，然后分别调用 RFID initialize 和 camera connect。没有要求二者相互等待，设备实现可并行工作。

identify() 先要求选舱，再调用 RFID。它没有检查 RFID Ready、序列是否活动或重复请求；当前 UI 未调用这个入口，因此问题尚未暴露。

第 142~162 行：开始批量识别

```
if (sequenceActive()) { /* 提示 */ return; }
QString clearError;
if (!m_sessions.clearAll(&clearError)) { /* 提示 */ return; }
m_captureSequenceAuthorized = false;
m_completedCaptureChambers.clear();
m_sequenceChambers = {4, 3, 2, 1};
m_sequenceIndex = 0;
m_sequenceMode = SequenceMode::Identifying;
m_sequenceStatus = "正在按 4 → 3 → 2 → 1 识别舱室";
emit stateChanged();
identifyNext();
```

- 活动序列互斥守卫在最前。
- **开始识别即清空所有旧绑定**；后续失败也不会恢复。这是当前批量识别用例的既定语义，重新识别同一 UID 时历史轮次仍会从数据库按 UID 加载。
- 新一轮识别同时清空 `m_completedCaptureChambers`，因此历史完成提示不会延续到新轮次。
- 只有队列和状态准备完整后才调用推进方法，避免同步回调看到半初始化状态。

第 164~179 行：推进识别

```
if (m_sequenceMode != SequenceMode::Identifying) return;
if (m_sequenceIndex >= m_sequenceChambers.size()) {
    m_captureSequenceAuthorized = std::any_of(... profile.has_value());
    finishSequence("四个舱室识别完成");
    return;
}
```

- 延时回调到达时先检查 mode，可安全忽略已结束序列。
- 队列结束后只要任一舱有 profile 就允许拍照。
- `finishSequence` 清空 mode，但识别完成不会清 authorized。

未结束时，取当前舱号、选择舱室、更新状态文本、通知 UI，再发起 recognize。选择先于识别，保证返回结果绑定到正确舱。

第 181~210 行：开始自动采集

```
if (sequenceActive()) return;
if (!m_captureSequenceAuthorized) return;
```

两个守卫分别保证无并行序列、拍照必须来自最近一次 RFID 识别授权。

```
m_sequenceChambers.clear();
for (int chamber : {4, 3, 2, 1})
    if (m_sessions.chambers().at(chamber - 1).profile)
        m_sequenceChambers.push_back(chamber);
m_completedCaptureChambers.clear();
```

只把识别成功的舱加入队列，保持指定顺序。`at()` 越界会断言，但固定 4 舱保证安全。

第 199~207 行清索引和本次完成集合、切模式、消耗授权、清暂停/预览/pending 状态并通知 UI。第 208 行启动预览，第 209 行立即尝试 `captureNext()`；没有预览帧时会进入等待状态。

第 212~252 行：推进自动采集

```
if (m_sequenceMode != SequenceMode::Capturing
    || m_capturePaused
    || m_captureRequestPending)
    return;
if (!m_previewFrameAvailable) {
    m_sequenceStatus = "等待 CCD 实时画面后开始拍照";
    emit stateChanged();
    return;
}
```

这四个条件构成 `capture` 的守卫集合。只有模式正确、未暂停、无在途请求且见过预览帧才继续。

队列为空时完成；正常情况下用 `m_sequenceIndex % size` 取舱。取模能防索引略大时越界，但也能掩盖错误索引并从头循环。

```
m_sessions.selectChamber(chamber);
if (!m_workflow.hasActiveRound()) {
    QString error;
    if (!m_workflow.createRound(&error)) {
        emit message(error, true);
        finishSequence();
        return;
    }
}
```

先切舱，再保证该舱有活动轮次。创建失败终止全序列。

```
m_captureRequestPending = true;
m_captureRequestDispatched = false;
m_discardPendingCaptureResult = false;
const quint64 requestToken = ++m_captureRequestToken;
QTimer::singleShot(150, this, [this, requestToken] {
    if (m_sequenceMode == SequenceMode::Capturing
        && !m_capturePaused
        && m_captureRequestPending
        && !m_captureRequestDispatched
        && requestToken == m_captureRequestToken) {
        m_captureRequestDispatched = true;
        m_camera->capture();
    }
});
```

`pending` 在定时器前置 `true`，阻止 150 ms 内预览帧重复排队。`token` 把延时回调与本次请求绑定：暂停取消后即使旧回调在恢复后到达，也不能再调用相机。真正调用 `capture()` 前先置 `dispatched`，暂停逻辑才能分辨“可直接取消”和“必须等待相机终局信号”的请求。

第 254~270 行：手工拍摄

自动序列活动时拒绝手工拍摄；已有请求正在保存时也拒绝。然后记录是否重拍、置 pending 并调用 camera。该入口没有先调用 `prepareCapture`，无活动轮次等错误要等图像已经拍回后才发现。

处理拍照结果：先过滤迟到帧，再提交业务数据

```
if (!m_captureRequestPending)
    return;
if (m_discardPendingCaptureResult) {
    m_captureRequestPending = false;
    m_captureRequestDispatched = false;
    m_discardPendingCaptureResult = false;
    if (m_sequenceMode == SequenceMode::Capturing && !m_capturePaused)
        QTimer::singleShot(0, this, [this] { captureNext(); });
    return;
}
m_captureRequestDispatched = false;
const int chamberBeforeCapture = m_sessions.selectedChamber();
```

先拒绝没有在途请求的迟到帧。暂停后已经下发的请求会保留 pending 锁，但其图像进入 discard 分支，只完成状态清理，绝不调用 `prepareCapture()`。这保证用户切换到浏览舱后，旧舱的相机结果不会写到新舱。

正常请求才清 dispatched，并记住返回时的舱，用于 commit 后确认上下文没有被信号副作用切换。

```
if (!isUsableCapture(image)) {
    m_captureRequestPending = false;
    emit message(..., true);
    if (m_sequenceMode == SequenceMode::Capturing)
        finishSequence();
    return;
}
```

无效图释放锁；自动模式终止，手工模式保留 workflow 供重试。

```
CaptureWorkflowService::PendingCapture pending;
if (!m_workflow.prepareCapture(image, m_captureRetake,
                                m_camera->exposure(), m_camera->gain(),
                                &pending, &error)) {
    /* 释放锁、提示、必要时终止 */
}
```

拍照完成后读取相机当前参数。严格审计场景应在发请求时冻结参数，否则返回前参数变化会记录错误值。

第 302 行在 commit 前把 pending 置 false。当前 commit 同步执行，不会处理事件循环，因此不会实际重入；若未来 commit 异步化，这个时机必须重新设计。

```

if (!m_workflow.commitCapture(pending, &commitError)) {
    emit message(commitError, true);
    if (m_sequenceMode == SequenceMode::Capturing)
        finishSequence();
    return;
}
m_captureRetake = false;

```

提交成功才清重拍标志。失败后手工模式保留旧值，下一次仍会按重拍处理，这可能是期望重试，也可能造成意外。

```

if (m_sequenceMode == SequenceMode::Capturing
    && m_sessions.selectedChamber() == chamberBeforeCapture) {
    const auto *model = m_sessions.selectedModel();
    const bool chamberCompleted = model && !model->rounds.isEmpty()
        && model->rounds.last().finished;
    if (chamberCompleted) {
        m_completedCaptureChambers.insert(chamberBeforeCapture);
        ++m_sequenceIndex;
    }
    if (m_completedCaptureChambers.size() == m_sequenceChambers.size()) {
        finishSequence(...);
        return;
    }
}

```

只有仍在自动模式且舱未变化才推进。完成一个舱的 16 孔后，舱号先写入本次完成集合，再把索引加一；未完成则继续同一舱的下一孔。QSet::insert() 具有幂等性，即使完成信号被重复观察，也不会重复增加完成数量。

提交后只在当前舱的活动轮次刚刚变为 finished 时，将 chamberBeforeCapture 插入 m_completedCaptureChambers 并推进索引；当集合大小等于当前队列大小才结束，否则 80 ms 后再次 captureNext()。这里刻意不检查“所有舱最后一轮是否 finished”，因为那会把历史轮次误当成本次序列结果。

第 326~343 行：完成/中止序列

```

const SequenceMode completedMode = m_sequenceMode;
if (completedMode == SequenceMode::Capturing && m_workflow.hasActiveRound())
    m_workflow.discardActiveRound();
if (completedMode == SequenceMode::Capturing)
    m_captureSequenceAuthorized = false;
m_sequenceMode = SequenceMode::None;
m_capturePaused = false;
m_captureRequestPending = false;
m_captureRequestDispatched = false;
m_discardPendingCaptureResult = false;
++m_captureRequestToken;
m_sequenceStatus.clear();
emit stateChanged();
if (!detail.isEmpty()) emit message(detail, false);

```

- 先保存旧 mode，因为后面要清空。
- 正常完成轮次不再 active；异常中止时 active 轮次被丢弃。

- 只要拍照序列结束，不论成功失败，都消耗授权。
- `m_previewFrameAvailable`、完成集合、队列和 `index` 没全部清空，但下次开始会重设本次序列需要的关键值；token 递增使尚未执行的延时拍摄回调失效，完成集合则保留供首页在序列结束后显示“本轮拍照完成”。
- 空 `detail` 表示异常路径已在上游提示，这里不重复消息。

第 345~368 行：暂停与恢复，隔离在途结果

只允许拍照模式暂停，且重复设置直接返回。暂停时若请求尚未下发，清 `pending` 并递增 token，立即取消延时请求；若相机已经收到 `capture`，则保留 `pending` 锁并设置 `discard` 标记。这样恢复不会抢先发出另一张图，必须等旧请求的 `captured/captureFailed` 到达并被消费。

恢复时用 0 ms `singleShot` 在下一次事件循环调用 `captureNext()`；仍有在途旧请求时守卫会阻止它继续。旧请求被丢弃并清锁后，控制器才会再次从 `m_sequenceIndex` 指定的原队列舱室继续。UI 查看其他舱室不会改变自动队列归属。

架构沉淀

控制器已经具备明确状态机雏形，并已处理页面切换造成的迟到帧竞态；自动拍摄是否符合无运动平台业务约束仍需按产品决策确认。另外要关注构造期信号丢失、同步保存，以及底层接口没有原生请求 ID 时的极端乱序场景。将状态转移整理成表并让设备信号携带请求 ID，会进一步提升可靠性。

7. `src/storage/database.h` 与 `database.cpp`

模块总览

`Database` 只负责 SQLite 连接生命周期和 schema，不负责领域查询。

头文件第 6~19 行：资源拥有者

```
class Database final {
public:
    Database() = default;
    ~Database();
    bool open(const QString &path, QString *error = nullptr);
    void close();
    bool isOpen() const { return m_db.isOpen(); }
    QSqlDatabase database() const { return m_db; }
    QSqlDatabase connection() const { return m_db; }
    static bool initializeSchema(QSqlDatabase db, QString *error = nullptr);
private:
    QSqlDatabase m_db;
    QString m_connectionName;
};
```

- `final` 表明这不是用于继承扩展的抽象。
- 析构调用 `close`，体现 RAII。
- `database()` 与 `connection()` 完全同义，存在重复 API。
- `QSqlDatabase` 是隐式共享连接句柄，按值返回不是新建连接。
- `schema` 初始化设为 `static`，测试可对现有连接单独调用。

实现第 7~24 行：打开数据库

```
namespace {  
void setError(QString *error, const QString &value) {  
    if (error) *error = value;  
}  
}
```

这个小工具统一可空错误输出，避免每处重复判断。

```
close();  
m_connectionName = QStringLiteral("tls401_%1")  
    .arg(static_cast<quintptr>(reinterpret_cast<quintptr>(this)));  
m_db = QSqlDatabase::addDatabase("QSQLITE", m_connectionName);  
m_db.setDatabaseName(path);  
if (!m_db.open()) {  
    setError(error, m_db.lastError().text());  
    close();  
    return false;  
}
```

- `open` 可重复调用，先关闭旧连接。
- 用对象地址生成进程内唯一连接名。
- 失败时立即 `close`，避免 Qt 全局连接表留下无效项。

```
QSqlQuery pragma(m_db);  
pragma.exec("PRAGMA foreign_keys=ON");  
pragma.exec("PRAGMA journal_mode=WAL");  
pragma.exec("PRAGMA busy_timeout=5000");  
return initializeSchema(m_db, error);
```

依次启用外键、WAL 和 5 秒锁等待。三个返回值均未检查；schema 失败时 `open()` 返回 `false`，但连接仍保持打开，调用者析构时才关闭。更对称的做法是 schema 失败立即 `close`。

第 27~34 行：关闭连接

```
if (m_connectionName.isEmpty()) return;  
if (m_db.isValid()) m_db.close();  
m_db = QSqlDatabase();  
QSqlDatabase::removeDatabase(m_connectionName);  
m_connectionName.clear();
```

关键顺序是先把成员句柄替换为空，再 `removeDatabase`。否则 Qt 会警告连接仍在被使用。外部 `Repository` 仍可能持有该连接句柄，所以控制器必须先 `delete Repository` 再 `delete Database`，当前析构顺序做到了。

第 36~50 行：建表和索引

`statements` 顺序先建被引用表，再建引用表，最后建索引：

- `tag_profile`：UID 主键、标签资料、本地姓名、首次/最后识别时间。
- `chamber_assignment`：舱号主键且限制 1~4；tag UID 唯一；外键引用标签。
- `capture_round`：自增 ID、标签、舱、轮次、状态和时间；同一 UID+轮次唯一。
- `capture_image`：图像元数据、相机参数、active/available、被替换图片引用。
- `idx_capture_image_active`：部分唯一索引，只约束 active=1 的记录。
- `idx_capture_round_active`：每个标签最多一个 in_progress 轮次。
- 普通索引优化按轮次/孔/时间读取。

循环中每条 DDL 独立执行，没有包 schema 事务。中途失败会留下部分创建状态，不过 `IF NOT EXISTS` 让下次可继续。

第 51~59 行：轻量迁移

```
PRAGMA table_info(capture_image)
```

遍历表列检查 `file_available` 是否存在；没有则 `ALTER TABLE ... ADD COLUMN`。这是向后兼容迁移，但只能处理一个版本。迁移数量增加后应维护 `schema_version` 并按版本顺序执行。

架构沉淀

连接生命周期和数据库约束设计总体稳健。要补强的是 PRAGMA/Schema 失败收尾、迁移版本化和 DDL 事务。

8. `src/storage/repository.h` 与 `repository.cpp`

模块总览

Repository 把 `TagProfile/ChamberModel/CaptureRound` 转成 SQL，并把查询行还原成领域对象。它是 SQL 的集中边界。

头文件第 6~25 行：能力分组

- 第 8~12 行：标签写入、舱室绑定与清理。
- 第 13~15 行：按全部舱室或单 UID 加载历史。
- 第 16~19 行：轮次编号、创建、完成、删除。
- 第 20~22 行：插入图像并处理替换关系。
- 第 24 行按值持有 Qt 数据库连接句柄，不拥有底层连接注册项。

`Repository final` 没有抽象接口， workflow 只能用真实类或空指针，难以注入内存仓储。产品化可提取 `IRepository`。

实现第 9~24 行：错误工具与缩略图解码

```
QImageReader reader(path);
const QSize sourceSize = reader.size();
if (!sourceSize.isValid()) return false;
reader.setScaledSize(sourceSize.scaled(QSize(800, 600), Qt::KeepAspectRatio));
*image = reader.read();
return !image->isNull();
```

先从文件头读尺寸，再要求解码器直接输出缩略图。与“先解完整大图再 scaled”相比，显著降低内存峰值。

第 25 行把传入连接移动到成员。QSqlDatabase 移动/复制都只是共享句柄，语义上表示 Repository 接管这个句柄变量。

第 27~33 行：标签 UPSERT

SQL 的 ON CONFLICT(uid) DO UPDATE 只更新 format、培养皿号、授精时间、女方姓名、病历号和 last_seen；**没有更新 male_name 和 first_seen_at**。这落实了“本地补录保留、首次识别不改”的规则。

第 30 行统一用 UTC ISO 毫秒字符串存库，避免本地时区变化。第 31 行按 ? 顺序绑定 9 个值：

1. UID；
2. 固定格式版本 1；
3. dishNumber 转 int；非数字会静默变 0；
4. 授精时间转 UTC；
5. 女方姓名；
6. 病历号；
7. 男方姓名空时绑定 SQL NULL；
8. 有效 identifiedAt，否则用 now；
9. last_seen now。

预编译和绑定值避免 SQL 注入，也正确处理引号和中文。

第 35~42 行：绑定舱室事务

```
if (chamberNo < 1 || chamberNo > 4 || !upsertTag(p, error))
    return false;
```

先校验舱号并保存标签。注意 UPSERT 发生在下面事务之前。

```
if (!m_db.transaction()) return false;
UPDATE chamber_assignment SET tag_uid=NULL ... WHERE tag_uid=?;
INSERT ... ON CONFLICT(chamber_no) DO UPDATE ...;
if (!q.exec() || !m_db.commit()) {
    m_db.rollback();
    ...
    return false;
}
```

- 第一个 UPDATE 确保该 UID 从任何旧舱解绑。
- 第二个 UPSERT 把目标舱绑定到新 UID，覆盖该舱旧标签。
- 二者在一个事务中，数据库不会出现 UID 同时两舱。
- UPSERT tag 不在同一事务，绑定失败时标签资料可能仍已更新。

第 44~65 行：清舱与清历史

`clearChamber()` 只把 assignment 的 UID/identified 置空，保留行和 updated_at；标签、轮次与图片历史都保留。

`clearCaptureHistory()` 在事务内先删 `capture_image`，再删 `capture_round`，顺序满足外键约束。它不删除磁盘图片，必须与 `ImageFileStore::clearCaptureStorage` 组成更高层用例，否则会产生孤儿文件。

第 67~72 行：加载舱室绑定

先查所有 assignment，再逐行按 tag_uid 查询 profile：

- 验证舱号 1~4。
- 把整数 dish_number 转回字符串。
- 用 ISODate 解析时间；写入使用 ISODateWithMs，Qt 能兼容解析。
- 写入 `(*chambers)[no - 1]` 前只验证 no，不验证传入 vector 至少 4 项，空数组会越界。
- tag 子查询失败被静默忽略，不会把错误返回给调用者。

第 74~104 行：按舱加载轮次

先查轮次，再对每轮查图像。只把 UID 与当前舱 profile 匹配的轮次挂入模型，防止历史舱号与当前绑定不一致时串数据。

```
CaptureRound round(round_no,
    status != QStringLiteral("in_progress"), id);
```

凡状态不是 in_progress 都当 finished，包括未知/failed/cancelled 状态。这种宽松映射可能把异常轮次显示成完成。

每张图片还原时间、active、available；数据库保存相对路径，读取时以数据库文件所在目录为根拼成绝对路径。

```
if (capture.available
    && (!QFileInfo::exists(absolutePath)
        || !loadPreviewImage(absolutePath, &capture.image))) {
    capture.available = false;
    UPDATE capture_image SET file_available=0 WHERE id=?;
}
```

如果文件丢失/损坏，内存标不可用并尝试修复 DB 元数据。UPDATE 失败不向上传播。

第 106~143 行：按 UID 加载轮次

逻辑与 `loadRounds()` 大量重复，区别是：

- 输入必须同时有非空 UID 和输出容器。
- 先 `rounds->clear()`，保证结果不与旧数据叠加。
- 查询不按舱筛选，因为历史跟随标签。
- 每轮仍执行一次图片查询，形成 N+1 查询。

这两段应提取私有 `loadImagesForRound`，减少修复时只改一处的风险。

第 145~154 行：下一个轮次号

```
SELECT COALESCE(MAX(round_no), 0) + 1
FROM capture_round
WHERE tag_uid=?
```

空历史时 MAX 为 NULL，COALESCE 转成 0，再加 1。并发创建仍可能得到相同编号，最终由 UNIQUE(tag_uid, round_no) 拒绝其中一个；应用没有自动重试。

第 156~167 行：创建与完成轮次

createRound() 插入 in_progress 记录并通过 lastInsertId() 返回自增主键。finishRound() 更新 status 和 finished_at，但没有检查 affected rows；不存在的 ID 也可能返回 true。

第 169~183 行：删除轮次

ID <= 0 被视为无需删除并返回 true。有效 ID 时在事务内先删图片行再删轮次行。虽然外键没有声明 ON DELETE CASCADE，显式顺序保证成功。

第 185~191 行：插入或替换图片

完整逻辑在一个事务内：

1. 若调用者未提供 replacedId，查询该轮/孔/焦层当前 active 图。
2. 找到旧图则设 is_active=0。
3. 插入新图，固定焦层 0、active=1、available=1，并记录 replaced_image_id。
4. 提交并返回新 ID。

这与部分唯一索引配合，确保任何时刻最多一张 active 图。当前工作流在数据库轮次创建前就处理重拍，最终每孔只 insert 一次，因此替换链能力尚未真正使用。

架构沉淀

Repository 很好地集中 SQL 与映射，但需要消除重复查询代码、修复输入容器边界、明确读取副作用，并把长单行 SQL/绑定拆开以便审查。

9. src/storage/imagefilestore.h 与 imagefilestore.cpp

模块总览

该模块只管理文件，不知道数据库表。它把图片从 QImage 变成有稳定目录结构的文件，并提供暂存、发布、清理能力。

头文件第 7~21 行：文件仓储接口

```
class ImageFileStore final {
public:
    explicit ImageFileStore(QString root = {});
    QString root() const { return m_root; }
    bool savePng(..., QString *relativePath, QString *error = nullptr) const;
    bool stageJpeg(..., QString *relativePath, QString *error = nullptr) const;
    bool finalizeStagedJpeg(..., QString *relativePath, QString *error =
nullptr) const;
    bool clearCaptureStorage(QString *error = nullptr) const;
    void remove(const QString &relativePath) const;
private:
    QString m_root;
};
```

- 所有写操作标为 `const`，含义是仓储对象配置不变，不代表外部文件系统无变化。
- 返回给 Repository 的都是相对路径，使 AppData 根目录迁移后数据库仍可用。
- `remove` 不返回失败，适合补偿清理，但无法反馈残留文件。

实现第 9~23 行：错误、UID 和文件名

```
QString sanitizedUid(QString uid)
{
    uid.replace(QRegularExpression("[^0-9A-Za-z_-]"), "_");
    return uid;
}
```

按值接收是有意的：在本地副本上替换，不修改调用者 UID。白名单只允许数字、英文字母、下划线、连字符，其余都变 `_`，阻止斜杠创建意外子目录。

```
return capturedAt.toLocalTime().toString("yyyyMMdd_HH:mm:ss_zzz")
    + "_" + QUuid::createUuid().toString(QUuid::WithoutBraces)
    + "_layer_00." + extension;
```

- 本地时间让人工查看目录更直观。
- 毫秒时间仍可能碰撞，UUID 提供额外唯一性。
- 文件名保留 `layer_00`，为未来焦层扩展预留协议。

第 25~43 行：临时写入再发布

```
if (!QDir().mkpath(QFileInfo(absolutePath).absolutePath())) {
    setError(error, "Cannot create image directory.");
    return false;
}
const QString temporary = absolutePath + ".tmp";
```

先递归建父目录，再在目标旁生成 `.tmp`。同目录 `rename` 通常具有原子替换语义。

```
if (!image.save(temporary, format, quality)
    || (QFileInfo::exists(absolutePath) && !QFile::remove(absolutePath))) {
    setError(...);
    QFile::remove(temporary);
    return false;
}
```

- 先完整写临时文件。
- 若目标已存在则先删除；正常 UUID 命名几乎不会冲突。
- 任一失败删除临时文件。

```
if (!QFile::rename(temporary, absolutePath)) {
    setError(...);
    QFile::remove(temporary);
    return false;
}
```

只有 rename 成功，外界才看见最终文件。若进程在 save 后、rename 前崩溃，会留下 `.tmp`，当前没有启动清理。

第 46~63 行：直接保存 PNG

构造函数移动 root 字符串到成员。

`savePng()` 先检查图像、UID、轮次和孔范围。然后生成：

```
images/{safeUid}/{yyyyMMdd}/round_{4位}/well_{2位}/{文件名}.png
```

`.arg(roundNo, 4, 10, '0')` 表示十进制至少 4 位、左侧补 0。第 60 行通过 `writeImage(..., "PNG", -1)` 使用 PNG 默认质量。第 61 行再次计算相对路径并把 Windows `\` 转 `/`，数据库路径跨平台更稳定。

第 65~80 行：暂存 JPEG

目录结构改为 `staging/`，格式为 JPG 质量 90。第 78 行直接返回 `rel`，它本身用 `/` 拼接。验证规则与 PNG 一致。

暂存用 JPEG 降低体积，但这意味着后面仅 copy 时正式图片已经发生有损压缩。

第 82~102 行：发布暂存图

验证参数后生成正式 `images/` 路径，检查源存在、创建目标目录、必要时删除冲突目标，最后 `QFile::copy(source, destination)`。

这里不立即删除源，原因是上层还没完成 16 次数据库插入；若后续失败，正式副本可删除而 staging 仍保留，方便统一补偿。全部成功后 workflow 才清 staging。

第 104~115 行：清空图片区域

循环处理 `images` 和 `staging`。路径存在但不是目录，或递归删除失败，都返回错误。它只在固定 root 子目录下操作，但该接口属于高风险运维能力，当前 UI 未接入。

第 117~120 行：删除单文件

空路径不处理，否则 `QDir(m_root).filePath(relativePath)` 后删除。没有检查删除结果，也没有验证 `../` 规范化后仍在 `root` 内。由于正常路径由本类生成，当前风险受控；如果未来接收外部 DB 或用户输入，应增加路径边界校验。

架构沉淀

该模块体现了稳定路径协议和临时文件发布模式。真正产品化时应统一最终格式、处理 `.tmp/staging` 崩溃残留，并让删除操作可观测。

10. `src/ui/mainwindow.h`

模块总览

头文件列出窗口的三页构建、刷新函数和 UI 显示态。业务实体由控制器持有，窗口只保存控件指针和回放索引。

第 6~16 行：前置声明

`Ui::MainWindow` 是 uic 生成类；其他 Qt 控件只以指针出现，用前置声明减少头文件依赖。`QVector` 通过 `applicationcontroller.h -> models.h` 间接可见，但直接使用的类型最好直接 `include`，避免脆弱的传递依赖。

第 18~24 行：舱室卡片视图引用

```
struct ChamberCardView {
    QFrame *frame = nullptr;
    QPushButton *select = nullptr;
    QLabel *state = nullptr;
    QLabel *details = nullptr;
    QVector<QPushButton*> wells;
};
```

这不是领域模型，而是把同一张首页卡片的控件引用打包。它避免维护 4 组平行数组。控件由 Qt parent 树拥有，结构体只观察，不负责 delete。

第 26~50 行：窗口行为

- `buildui` 组装三页；三个 `build*Page` 各自返回 `QWidget`。
- `refreshAll` 按当前页分派；三个 `refresh*` 做状态投影。
- `showPage/showMessage` 管导航和提示。
- `stateText/chamberSummary/chamberStateText` 负责格式化 View Model 文本。
- `displayedDishRound`、两种 `playback` 方法负责从领域数据选出当前要显示的图。
- `setDishRoundIndex` 集中维护轮次滑块与显示索引。
- `updatePreview` 只更新校准实时图。
- `selectChamberForInspection` 是查看性切舱的统一安全入口：识别中拒绝切换，自动拍照中先暂停再切换。

第 51~81 行：成员分类

- 第 51~59 行：控制器、uic 对象、顶层页面和主要 Label。
- 第 60~69 行：首页/皿页/孔页的重复控件集合、时间轴控件和 `m_dishPlayButton`。保存播放按钮指针，才能在导航停止定时器时同步恢复图标。
- 第 70~73 行：当前孔、历史索引、皿轮次索引和播放步长，是纯显示态。
- 第 74~75 行：孔回放与皿回放两个独立定时器。
- 第 76~79 行：孔详情浏览/校准模式控件和首页拍照按钮。
- 第 80 行当前页索引，约定 0 首页、1 皿详情、2 孔详情。裸整数可改枚举增强可读性。

架构沉淀

窗口没有复制业务模型，这是正确的；但它暴露出页面拆分信号：当一个类需要保存三页所有控件引用时，应考虑每页独立组件。

11. `src/ui/mainwindow.ui`

模块总览

`.ui` 只提供稳定外壳：窗口、顶栏、标题、状态和页面容器。具体页面在 C++ 动态生成。

第 1~8 行：窗口元数据

XML 声明 Qt Designer UI 4.0 格式，类名必须与 `Ui::MainWindow` 匹配。默认几何是 1280x820，窗口标题为联动演示名称；它是初始尺寸，不是固定尺寸。

第 9~31 行：外壳布局

`centralWidget` 使用垂直布局且四边距、间距为 0。第一项是最小高 56 的 `topBar`，内部水平布局：左标题、中间可扩展 `spacer`、右状态。第二项是 `QStackedWidget` `pages`，它占用剩余空间并由 C++ 加入三页。

第 32~37 行：闭合、资源和连接

UI 文件没有直接声明资源与信号连接；资源由 `qrc/C++` 使用，连接全部在 `MainWindow` 构造和 `build` 函数中完成。

架构沉淀

“Designer 外壳 + C++ 动态页”适合快速 Demo，但视觉维护者难以在 Designer 中看到完整页面。长期应选择统一策略：页面各自 `.ui`，或完全代码化并组件化。

12. `src/ui/mainwindow.cpp`

模块总览

该文件包含控件工厂、自定义孔缩略图、三页构建、回放定时器、导航和状态刷新。读法应分成“构建一次”和“状态变化时反复刷新”两条线。

第 1~25 行：Qt 控件依赖

引入按钮组、`QByteArray`、下拉框、布局、绘图、滑块、样式、定时器等。`QByteArray` 用于把十六进制 UTF-8 字节还原为“本轮拍照完成”，避开当前 MSVC/Qt 源码中文字面量的编码差异。

`QDoubleSpinBox` 和 `QIcon` 当前没有使用，属于可清理依赖；`QSpinBox` 用于校准占位控件。

第 26~40 行：控件工厂

```
QLabel *label(const QString &text = {}) {
    auto *result = new QLabel(text);
    result->setWordWrap(true);
    return result;
}

QPushButton *button(const QString &text, bool primary = false) {
    auto *result = new QPushButton(text);
    if (primary) result->setObjectName("primary");
    return result;
}
```

两个匿名命名空间函数减少动态 UI 的重复代码。控件创建时没有 parent，但加入布局后 Qt 会重新设置所属关系。primary 通过 objectName 让 QSS 应用主按钮样式。

第 42~85 行：自绘孔缩略图按钮

构造函数保存孔号。`setThumbnail()` 的逻辑是：空图清缓存；非空图取短边，以中心正方形裁剪，再转成 `QPixmap` 缓存，最后 `update()` 请求重绘。

`paintEvent()` 没调用父类实现，而是：

1. 用 `initStyleOption` 取得当前 hover/pressed/checked 等状态。
2. 清掉默认文字和图标。
3. 让当前 Style 绘制标准按钮底板。
4. 在顶部绘制孔号。
5. 有缩略图时，在余下区域等比例画正方形图。

这样既保留 QSS 状态，又完全控制内容。硬编码颜色 `#d7e8f5` 绕过主题层，换主题时可能不协调。

第 87~101 行：辅助转换

`setWellThumbnail()` 用 `dynamic_cast` 确认传入按钮是自定义类型后设置缩略图。由于容器类型是 `QPushButton*`，这里需要向下转型。

`wellStateText()` 把四种孔状态转中文，但当前文件没有调用该函数，是死代码。

第 104~136 行：窗口构造、析构和两个播放定时器

```
ui->setupUi(this);
buildUi();
connect(controller message/state, ...);
connect(sessions changed, ...);
connect(workflow changed, ...);
connect(camera previewFrame, updatePreview);
```

先由 `uic` 创建外壳，再动态添加页面，最后连接状态信号。会话 `changed` 可能经 workflow 再次 `emit changed`，窗口因此在一次变化中刷新多次；当前数据量小，影响有限。

孔回放定时器每 500 ms：重新计算当前孔可播放数量，有帧则按 `m_playStep` 取模推进。皿回放定时器也每 500 ms：有轮次则循环调用 `setDishRoundIndex`。

定时器以窗口为 `parent` 自动销毁。析构只需 `delete ui`；动态控件都在 Qt 对象树中。

第 139~147 行：添加三页

把 `.ui` 中的标题、状态、`pages` 缓存到成员，然后依次添加首页、皿页、孔页。添加顺序定义了整个文件使用的页面整数协议。

第 149~231 行：首页构建

首页使用垂直布局，主体是 2x2 舱室卡片网格。每舱循环创建：

- 可勾选舱号按钮；
- 状态 `Label`；
- 居中的概要 `Label`；
- 2x8 共 16 个 `wellThumbnailButton`。

孔缩略图点击 `Lambda` 捕获 `chamberNo`：先通过 `selectChamberForInspection()` 处理导航安全，再检查 `profile`；有数据进入皿详情，无数据提示。这意味着首页孔按钮是“进入该舱皿详情”，并不直接进入所点击的具体孔，`Lambda` 没捕获 `well`。

舱号按钮同样通过该安全入口切舱。识别队列运行时入口拒绝切换，自动拍照运行时入口先暂停，再允许用户浏览目标舱；所有控件引用存到 `m_homeCards`，后续 `refreshHome` 更新。

底部动作区创建“开始识别”和“开始拍照”，分别调用控制器批量用例。UI 不直接循环舱室或设备，这是正确边界。

第 233~339 行：皿详情构建

页面水平分三块：

1. 140 px 舱室导航；
2. 225 px 患者/培养皿资料与返回按钮；
3. 自适应 4x4 孔图和轮次时间轴。

皿详情左侧舱室按钮也统一走安全切舱入口，是否可用由 `refreshDish` 根据 `profile` 决定。

16 个孔图加入 `QButtonGroup` 并赋 ID，但后续没有使用 `group` 信号，只保留了按钮数组。点击某孔会设置 `m_detailWell=i`、历史索引归零并进入孔详情。

轮次控制：

- `previous/next` 调整 `m_dishRoundIndex`，`qBound` 会在两端停住，不循环。
- `play` 按钮保存到 `m_dishPlayButton`，启动时显示 `||`、停止时显示 `>`；定时器内部按轮次数取模循环。保留成员指针是为了离开页面时也能同步复位图标。
- 速度 1X~4X 映射 500/250/150/100 ms；默认 `ComboBox` 选 3X，但连接发生在 `setCurrentIndex(2)` 之后，所以定时器初始仍是构造时的 500 ms，直到用户改变一次速度。这是初始化顺序 bug。
- `Slider` 的 `valueChanged` 统一调用 `setDishRoundIndex`。

第 341~504 行：孔详情与校准页构建

左侧 2x8 的 16 孔选择按钮；中间患者信息和浏览/校准模式；右侧是 `m_wellModes` 中的浏览页或校准页。

浏览页创建大图、上一张/播放/下一张、速度、轮次标签和滑块：

- 上一/下一按历史数量取模循环。
- 播放按钮切定时器并把文本从符号改为中文“暂停/播放”，造成按钮宽度固定 34 时文本可能放不下。
- 3x/4x 把 `m_playStep` 设为 2，即每次跳过一帧；这不仅加快时间间隔，还会漏播轮次。更直观的速度实现只调整 `interval`。
- 滑块变化时检查是否不同，防止无意义刷新；refresh 中还会用 `QSignalBlocker`。

校准页创建 X/Y/Z/L/曝光五行：每行左右按钮只调用 `QSpinBox::stepDown/stepUp`。保存按钮没有 connect，步长 preset 也没有 `ButtonGroup` 或行为；曝光值不会调用相机 `setExposure`。所以这部分只是界面原型。

模式切换：

- 浏览模式取消拍照暂停并启动预览。
- 校准模式暂停自动拍照并启动预览。
- 两个 checkable 按钮手工互斥，未使用 `QButtonGroup exclusive`。

第 507~515 行：按当前页刷新

```
if (m_currentPage == 0) refreshHome();
else if (m_currentPage == 1) refreshDish();
else refreshWell();
```

任何非 0/1 值都当孔详情。状态变化时只刷新可见页，避免不必要的图片缩放，但隐藏页控件可能暂时陈旧，进入页面时 `showPage` 会再次刷新。

第 517~551 行：文本格式化

`stateText()` 把设备四态转成中文。`chamberSummary()` 无 profile 返回“未绑定”，有 profile 时显示培养皿、女方、病历、发育天数和 `rounds.size()`。这里把未完成内存轮次也计为“已采集轮次”，文字可能偏乐观。

`chamberStateText()` 按优先级推断：识别中 > 空舱 > 当前舱活动轮次 > 本次序列完成集合 > 未完成轮次 > 等待本轮拍照。它不再把历史 `last().finished` 直接显示成“本轮完成”，因为历史轮次可能来自上次运行。完成文案通过 `QString::fromUtf8(QByteArray::fromHex(...))` 构造，解决源码中文字符串在当前编译链下的乱码。

第 553~611 行：选择展示轮次与回放数据

`displayedDishRound()`：无数据返回 null；否则索引小于 0 时默认最新一轮，再用 `qBound` 限制范围。

`playbackForWell()`：逐轮检查该孔，从后向前找第一张 `active && available`，每轮最多放一张。`playbackRoundsForWell()` 用同样遍历返回对应轮次号。两个方法必须同步维护，否则图和轮次标签会错位。

`setDishRoundIndex()`：无轮次时置 -1；有轮次时夹紧索引。如果滑块值不同，用 `QSignalBlocker` 暂停信号再设值，避免递归调用自身；最后刷新皿页。

第 613~660 行：首页刷新

先读取四舱、当前舱、活动轮次和当前孔。拍照按钮只由 `canStartCapture()` 控制，并设置解释性 Tooltip。

每张卡：

- checked 映射当前舱；
- state/details 由格式化方法产生；
- 动态属性 `activity` 按 recognizing、capturing、idle 三态设置，分别对应蓝、绿、灰；改变后 unpolish/polish，强制 QSS 重匹配；
- 所有孔先重置 empty 和空图；
- 最近一轮有可用图则设 complete 和缩略图；
- 当前活动孔覆盖为 current；
- 再刷新每个按钮样式。

顶部状态合并 RFID、CCD、数据库、固定拍摄顺序和当前 `sequenceStatus`。`showMessage()` 也写同一个 Label，下一次 `refreshHome()` 会覆盖刚显示的消息，因此错误提示不是持久消息队列。

第 662~694 行：皿详情刷新

根据当前模型设置轮次滑块范围。只有 `m_dishRoundIndex < 0` 才默认最新；切换到另一个舱时旧索引保留。

资料文本用多次 `QString::arg` 填入舱号、姓名、发育天数、病历号、培养皿号、授精时间和培养小时数。长模板难以维护，字段顺序一旦改动容易错位，适合提取 ViewModel/格式化函数。

四个舱按钮仅在有 profile 时启用并同步 checked。

第 680 行取得 `wellStates()` 但后续没有使用，是无效计算。孔按钮 checked 使用工作流 `currentWell`，即使当前显示的是历史轮次也会显示活动选择；缩略图来自 `displayedDishRound()`。

第 696~735 行：单孔刷新

先同步 16 个孔导航按钮，然后生成患者/培养皿详情。

无历史时：标签显示暂无轮次，滑块在 `QSignalBlocker` 中复位并禁用，大图清 pixmap 后显示文字，然后提前返回。

有历史时：

- 用 `qBound` 修正历史索引。
- 从平行 rounds 数组取得真实轮次号。
- `QSignalBlocker` 中设置滑块范围和值。
- 把当前 `QImage` 转 `QPixmap`，并按当前 Label 尺寸平滑缩放。

窗口 `resize` 后如果没有再次 `refresh`，pixmap 不会适应新尺寸；可在 `resizeEvent` 中重算或让自定义控件 `paint` 时缩放。

第 737~779 行：页面导航、安全切舱与业务副作用

- 离开页面停止回放，并把 `m_displayButton` 文本恢复为 `>`，保证图标与定时器真实状态一致。
- 从孔详情离开时恢复自动拍摄。
- 进入孔详情时暂停自动拍摄、启动预览，并强制回浏览模式。
- 回首页时保持/启动预览，注释说明避免采集线程停启竞态。
- 设置 `QStackedWidget` 页、记录索引、更新标题并刷新。

`selectChamberForInspection()` 是查看性舱室切换的统一入口：RFID 批量识别中拒绝切换，避免识别结果绑定到错误舱；自动拍照中先暂停，再改变 `selected chamber`。配合控制器的迟到帧丢弃，返回后会按原自动队列继续。

导航函数仍直接暂停/恢复业务序列，因此“打开详情”不仅是视觉动作。后续路由变复杂时，这种副作用宜集中到导航控制器或显式用例。

第 781~792 行：消息与实时预览

`showMessage()` 忽略空文本，把错误前缀为“提示”、成功前缀为“完成”，写入顶栏状态 `Label`。

`updatePreview()` 只有当前在孔详情且校准子页可见时才更新，使用 `FastTransformation` 降低高频预览开销。浏览页展示历史图，不展示实时帧。

架构沉淀

`MainWindow` 已基本遵守“事件向下、状态向上”的单向流，但页面职责过多、校准功能未闭环、回放初始化有细节 bug。拆页组件后，每页只订阅自己需要的 `ViewModel`，会更容易测试和维护。

13. `src/main.cpp`

模块总览

入口只做应用生命周期和顶层对象装配。

第 1~5 行：依赖

引入控制器、窗口、`QApplication`、`QFile` 和 `QTimer`。业务入口不直接包含设备真实类或 `Repository`，它们由控制器内部组装。

第 7~16 行：启动顺序

```
int main(int argc, char *argv[]) {
    QApplication app(argc, argv);
```

必须先创建 `QApplication`，再创建任何 `QWidget/QPixmap` 相关对象。它还负责解析 Qt 参数和事件循环。

```
QFile style(QStringLiteral(":/styles/default.qss"));
if (style.open(QIODevice::ReadOnly))
    app.setStyleSheet(QString::fromUtf8(style.readAll()));
```

`:/` 表示 Qt Resource System，不是磁盘绝对路径。QSS 以 UTF-8 解码后应用到整个应用；失败静默使用默认样式。

```
ApplicationController controller;
MainWindow window(&controller);
window.show();
```

控制器先构造，窗口持有非 owning 指针。局部变量逆序析构，所以退出时先销毁窗口，后销毁控制器，生命周期安全。

```
QTimer::singleShot(150, &controller,
                  &ApplicationController::initializeDevices);
return app.exec();
```

- 延迟 150 ms 让窗口先显示并进入事件循环。
- 带 context 的 singleShot 在 controller 被销毁时自动取消。
- `app.exec()` 阻塞运行事件循环，关闭最后窗口后返回退出码。
- 延时不等于异步线程；设备方法若同步耗时，150 ms 后仍会卡 GUI。

架构沉淀

入口保持很干净。更成熟的 Composition Root 会在这里创建 Repository 与接口实现，再通过构造函数注入 Controller，从而让控制器不依赖具体设备类。

14. 把逐行理解转化为自己的能力

建议按以下断点路线亲自验证：

1. RFID: `identifyAll -> identifyNext -> recognized Lambda -> bindProfile -> assignChamber`。
2. 单张图: `captureNext -> camera captured -> prepareCapture -> commitCapture -> stageJpeg`。
3. 第 16 张: `commitCapture -> persistCompletedRound -> createRound -> insertImage x16 -> finishRound`。
4. UI: `workflow changed -> refreshAll -> refreshHome/refreshDish/refreshwell`。
5. 异常：让第 8 张暂存失败、让第 16 张 SQL 失败、拍照后立即切页，观察哪一层清 pending、哪一层删 staging、哪一层提示。

读完代码只是“知道”。能够在不看原实现的情况下画出状态机、解释每个持久化补偿步骤，并说明“同 UID 重识别恢复历史”“本次完成集合”“暂停后丢弃在途结果”三者各自解决什么问题，才是把它真正转化成自己的知识。